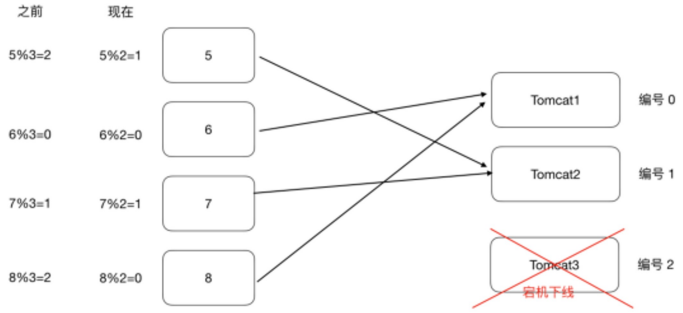
分布式集群架构-场景化解决方案

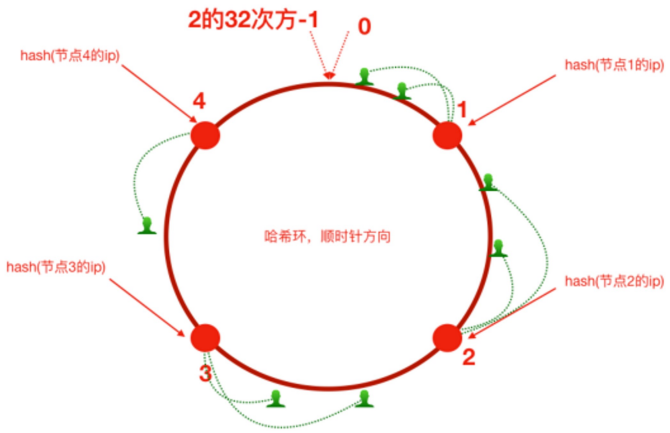
2020年8月15日 8:38

1. 一致性Hash算法
    a. Hash 算法：
        i. 应用：数据存储和查找领域
        ii. 为什么使用Hash：
            栗子：
                判断某个数n是否在数列中？
                顺序查找法：也就是循环遍历；
                二分查找；
                直接寻址法：就是把数据和数组的下标绑定在一起，查找时可以直接通过arr[n]获取；
                优点：查找速度快；
                缺点：
                浪费空间；
                数据重复时，导致空间不足；
                Hash：比如通过对数据求模（数据%空间位置数），就是简单的Hash算法；
                存在问题：Hash冲突：
                解决：
                开放寻址法：冲突时向前或向后找空闲位置存放，但长度不能扩展，比如10个位置来了11个数据，就莫得办法；
                拉链法：在元素存储位置存放一个链表；不过hash算法设计的比较菜的话，查询效率就很低；

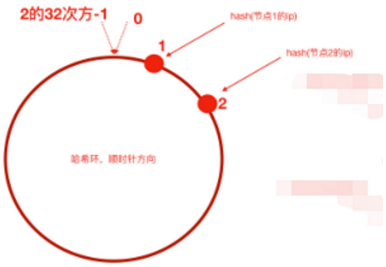
        iii. 主要场景：
            1) 请求的负载均衡：比如nginx的ip_hash策略；
            2) 分布式存储：比如使用redis集群进行数据存储时，锁定数据的存储节点；
        iv. 存在问题：
            down机时，所有hash都需要计算；



    b. 一致性Hash：
        i. 思路：
            假设有一条直线，首尾分别定为0和2的32次方减1，然后将其首尾相连，形成闭环，这个环就叫hash环；
            那么，它是如何解决的路由的问题的呢？
            首先，对服务器IP进行hash对应到hash环上，就是（key=hash环的值，value=ip）；
            接着对客户端ip进行hash对应到hash环上，然后顺时针方向查找最近的服务器节点；

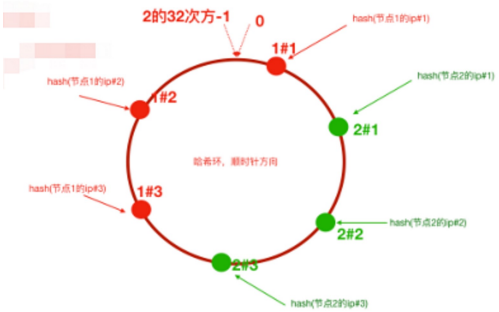


        ii. 存在问题：服务节点太少时，会出现数据倾斜



解决办法：引入虚拟节点

具体做法就是为服务器IP后面增加编号来实现，比如为每台服务器计算三个虚拟节点，于是可以分别计算‘ip1#1’，‘ip1#2’，‘ip1#3’，‘ip2#1’，‘ip2#2’，‘ip2#3’的hash，形成6个虚拟节点；那客户端被路由到虚拟节点时，其实就是被路由到虚拟节点对应的真实节点；



iii. 代码实现：

```
public class ConsistentHashNoVirtual {
  public static void main(String[] args) {
    //step1 初始化：把服务器节点IP的哈希值对应到哈希环上
    // 定义服务器ip
    String[] tomcatServers = new String[]{"123.111.0.0", "123.101.3.1", "111.20.35.2", "123.98.26.3"};

    SortedMap<Integer, String> hashServerMap = new TreeMap<>();

    for (String tomcatServer : tomcatServers) {
      // 求出每一个ip的hash值，对应到hash环上，存储hash值与ip的对应关系
      int serverHash = Math.abs(tomcatServer.hashCode());
      // 存储hash值与ip的对应关系
      hashServerMap.put(serverHash, tomcatServer);
    }

    //step2 针对客户端IP求出hash值
    // 定义客户端IP
    String[] clients = new String[]{"10.78.12.3", "113.25.63.1", "126.12.3.8"};
    for (String client : clients) {
      int clientHash = Math.abs(client.hashCode());
      //step3 针对客户端,找到能够处理当前客户端请求的服务器（哈希环上顺时针最近）
      // 根据客户端ip的哈希值去找出哪一个服务器节点能够处理（）
      SortedMap<Integer, String> integerStringSortedMap = hashServerMap.tailMap(clientHash);
      if (integerStringSortedMap.isEmpty()) {
        // 取哈希环上的顺时针第一台服务器
        Integer firstKey = hashServerMap.firstKey();
        System.out.println("==========>>>>客户端：" + client + " 被路由到服务器：" + hashServerMap.get(firstKey));
      } else {
        Integer firstKey = integerStringSortedMap.firstKey();
        System.out.println("==========>>>>客户端：" + client + " 被路由到服务器：" + hashServerMap.get(firstKey));
      }
    }
  }
}
```

c. Nginx配置一致性Hash负载均衡策略

2. 集群服务器时间同步问题

思路：

1) 分布式集群中各个服务器可以连接互联网：从国家授时中心同步时间

```
#使用 ntpdate 网络时间同步命令
ntpdate -u ntp.api.bz  #从一个时间服务器同步时间
```

2) 分布式集群中某一个节点可以访问互联网或者都不能访问互联网：选取其中一台服务器作为时间服务器，其他服务器从该服务器同步时间

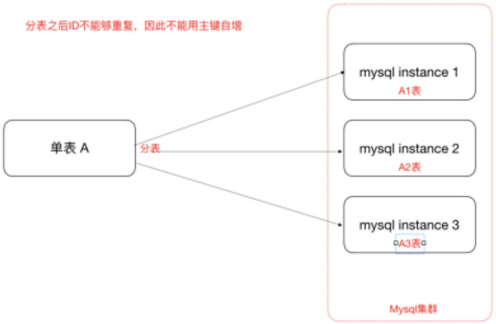
- 首先设置好A的时间
- 把A配置为时间服务器（修改/etc/ntp.conf文件）

```
1、如果有 restrict default ignore，注释掉它
2、添加如下几行内容
   restrict 172.17.0.0 mask 255.255.255.0 nomodify notrap  # 放开局域网同步功能,172.17.0.0是你的局域网网段
   server 127.127.1.0 # local clock
   fudge 127.127.1.0 stratum 10
3、重启生效并配置ntpd服务开机自启动
   service ntpd restart
   chkconfig ntpd on
```

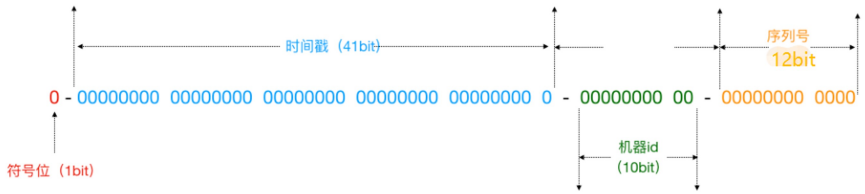
- 集群中其他节点就可以从A服务器同步时间了

```
ntpdate 172.17.0.17
```

3. 分布式ID（分布式集群环境下的全局唯一ID）解决方案
   a. 为什么需要分布式ID？



   b. 解决方案：
      i. UUID：Universally Unique Identifier，通用唯一识别码；
      ii. 独立数据库的自增ID：
         说明：单独使用某个数据库的某张表，将其ID设置为自增；然后通过MySQL的select last_insert_id()获取这张表自增生成的ID；
         存在问题：性能和可靠性无法保证；
      iii. SnowFlake 雪花算法：
         1）原理：基于这个算法可以生成一个8个字节即64bit的long型的ID；



说明：
符号位：0代表正数，1代表负数；
时间戳：表示某一个毫秒；
机器ID：代表当前算法运行机器的ID；
序列号：代表同一个机器同一个毫秒可以生成的ID序号；

衍生方案：
滴滴的tinyid（基于数据库实现）、百度的uidgenerator（基于SnowFlake）、美团的leaf（基于数据库和SnowFlake）；

      iv. 借助Redis的Incr命令获取全局唯一ID；

```
Jedis jedis = new Jedis("127.0.0.1",6379);
try {
  long id = jedis.incr("id");
  System.out.println("从redis中获取的分布式id为: " + id);
} finally {
  if (null != jedis) {
    jedis.close();
  }
}
```

4. 分布式调度（分布式环境下实现定时任务）问题

a. 定时任务的场景：

- 订单审核、出库
- 订单超时自动取消、支付退款
- 礼券同步、生成、发放作业
- 物流信息推送、抓取作业、退换货处理作业
- 数据积压监控、日志监控、服务可用性探测作业
- 定时备份数据
- 金融系统每天的定时结算
- 数据归档、清理作业
- 报表、离线数据分析作业

b. 什么是分布式调度？（2层含义）

i. 运行在分布式集群环境下的调度任务（同一个定时任务程序部署多份,只应该有一个定时任务在执行）；

ii. 分布式调度->定时任务的分布式->定时任务的拆分(即为把一个大的作业任务拆分为多个小的作业任务,同时执行)；

c. 定时任务与消息队列的区别

i. 共同点：

1) 异步处理：

比如注册、下单事件。

2) 应用解耦：

3) 流量削峰：

ii. 不同点：

定时任务是时间驱动的，而消息队列是事件驱动的；

定时任务更倾向于批处理，而消息队列趋向于逐条处理；

d. 定时任务的实现方式

i. 任务调度框架Quartz

```
* cron表达式由七个位置组成，空格分隔
  * 1、Seconds（秒）  0~59
  * 2、Minutes（分）  0~59
  * 3、Hours（小时）  0~23
  * 4、Day of Month（天）1~31,注意有的月份不足31天
  * 5、Month（月） 0~11,或者 JAN,FEB,MAR,APR,MAY,JUN,JUL,AUG,SEP,OCT,NOV,DEC
  * 6、Day of Week(周) 1~7,1=SUN或者  SUN,MON,TUE,WEB,THU,FRI,SAT
  * 7、Year（年）1970~2099  可选项
```

ii. pom引用

```
<dependency>
        <groupId>org.quartz-scheduler</groupId>
        <artifactId>quartz</artifactId>
        <version>2.3.2</version>
</dependency>
```

5. 分布式调度框架Elastic-Job

a. 简介：当当网开源的分布式调度解决方案，基于Quartz二次开发，由两个子项目组成Elastic-Job-Lite和Elastic-Job-Cloud；

b. 主要功能：

i. 分布式调度协调：在分布式环境中,任务能够按指定的调度策略执行,并且能够避免同一任务多实例重复执行；

ii. 丰富的调度策略：基于成熟的定时任务作业框架Quartz cron表达式执行定时任务；

iii. 弹性扩容缩容：当集群中增加某一个实例,它应当也能够被选举并执行任务;当集群减少一个实例时,它所执行的任务能被转移到别的实例来执行；

iv. 失效转移：某实例在任务执行失败后,会被转移到其他实例执行；

v. 错过执行作业重触发：若因某种原因导致作业错过执行, 自动记录错过执行的作业,并在上次作业完成后自动触发；

vi. 支持并行调度：支持任务分片,任务分片是指将一个任务分为多个小任务项在多个实例同时执行；

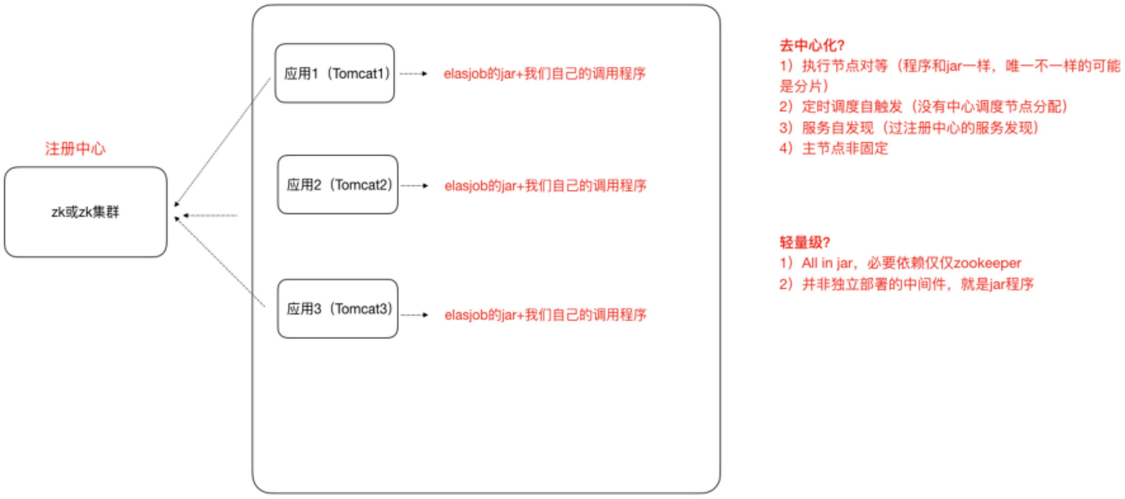
vii. 作业分片一致性：当任务被分片后,保证同一分片在分布式环境中仅一个执行实例；

c. pom引用：

```
<dependency>
        <groupId>com.dangdang</groupId>
        <artifactId>elastic-job-lite-core</artifactId>
        <version>2.1.5</version>
</dependency>
```

另外，Elastic-Job依赖于Zookeeper进行分布式协调；

d. 轻量级与去中心化：



e. 任务分片：

Elastic-Job可以把作业分为多个的task (每一个task就是一个任务分片) ,每一个task交给具体的一个机器实例去处理(一个机器实例是可以处理多个task的) ,但是具体每个task执行什么逻辑由我们自己来指定。
Strategy策略定义这些分片项怎么去分配到各个机器上去,默认是平均去分,可以定制,比如某一个机器负载比较高或者预配置比较高,那么就可以写策略。分片和作业本身是通过一个注册中心协调的,因为在分布式环境下,状态数据肯定集中到一点,才可以在分布式中沟通。

```java
public class ElasticJobMain {

    public static void main(String[] args) {
        // 配置分布式协调服务（注册中心）Zookeeper
        ZookeeperConfiguration zookeeperConfiguration = new ZookeeperConfiguration( serverLists: "localhost:2181",  namespace: "data-archive-job");
        CoordinatorRegistryCenter coordinatorRegistryCenter = new ZookeeperRegistryCenter(zookeeperConfiguration);
        coordinatorRegistryCenter.init();

        // 配置任务（时间事件、定时任务业务逻辑、调度器）
        JobCoreConfiguration jobCoreConfiguration = JobCoreConfiguration
                .newBuilder( jobName: "archive-job",  cron: "*/2 * * * * ?",  shardingTotalCount: 3)
                .shardingItemParameters("0=bachelor,1=master,2=doctor").build();
        SimpleJobConfiguration simpleJobConfiguration = new SimpleJobConfiguration(jobCoreConfiguration, ArchivieJob.class.getName());

        JobScheduler jobScheduler = new JobScheduler(coordinatorRegistryCenter, LiteJobConfiguration.newBuilder(simpleJobConfiguration).overwrite(true).build());
        jobScheduler.init();
    }
}
```

```java
public class ArchivieJob implements SimpleJob {
    @Override
    public void execute(ShardingContext shardingContext) {
        int shardingItem = shardingContext.getShardingItem();
        System.out.println("=====>>>>当前分片：" + shardingItem);

        // 获取分片参数
        // 0=bachelor,1=master,2=doctor
        String shardingParameter = shardingContext.getShardingParameter();

        // 1 从resume表中查询出1条记录（未归档）
        String selectSql = "select * from resume where state='未归档' and education='" + shardingParameter + "' limit 1";
        List<Map<String, Object>> list = JdbcUtil.executeQuery(selectSql);
        if (list == null || list.size() == 0) {
            System.out.println("数据已经处理完毕！！！！！！");
            return;
        }
        // 2 "未归档"更改为"已归档"
        Map<String, Object> stringObjectMap = list.get(0);
        long id = (long) stringObjectMap.get("id");
        String name = (String) stringObjectMap.get("name");
        String education = (String) stringObjectMap.get("education");

        System.out.println("=======>>>>id：" + id + "  name：" + name + " education：" + education);

        String updateSql = "update resume set state='已归档' where id=?";
        JdbcUtil.executeUpdate(updateSql, id);

        // 3 归档这条记录，把这条记录插入到resume_bak表
        String insertSql = "insert into resume_bak select * from resume where id=?";
        JdbcUtil.executeUpdate(insertSql, id);
    }
}
```

f. 弹性扩容：

i. 分片项也是一个OB配置,修改配置,重新分片,在下一次定时运行之前会重新调用分片算法,那么这个分片算法的结果就是:哪台机器运行哪一个一片,这个结果存储到zk中的,主节点会把分片给分好放到注册中心去,然后执行节点从注册中心获取信息(执行节点在定时任务开启的时候获取相应的分片)。

ii. 如果所有的节点挂掉值剩下一个节点,所有分片都会指向剩下的一个节点,这也是Elasticjob的高可用。

6. Session共享问题

a. 场景：Session丢失问题分析（nginx默认轮询策略）

b. 解决Session一致性的方案

i. Nginx的IP_Hash策略（可以用）：

同一个客户端IP的请求都会被路由到同一个目标服务器，也叫做会话粘滞；

a) 优点：配置简单，不入侵应用，不需要额外修改代码

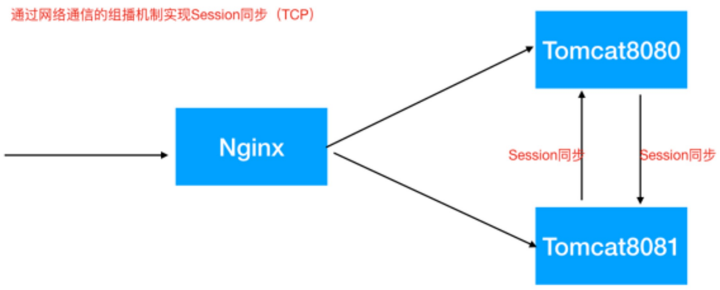
b) 缺点：服务器重启Session丢失；存在单点负载高的风险；单点故障问题；

ii. Session复制（不推荐）：

多个tomcat之间通过修改配置文件，达到Session之间的复制；

a) 优点：不入侵应用；便于服务器水平扩展；能适应各种负载均衡策略；服务器重启或者宕机不会造成Session丢失；

b) 缺点：性能低；内存消耗；不能存储太多数据，否则数据越多越影响性能；延迟性；

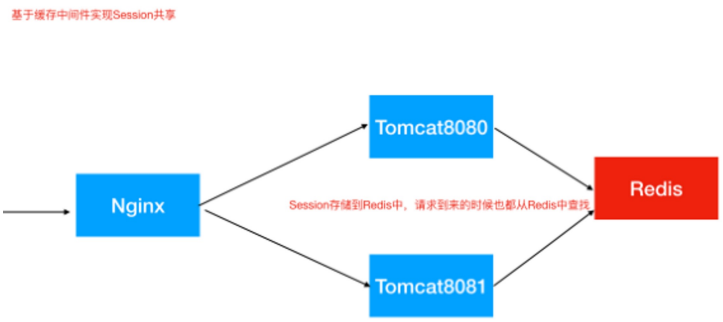


iii. Session共享，Session集中存储（推荐）：

Session的本质就是缓存，将Session数据专业的缓存中间件，比如Redis；

a) 优点：能适应各种负载均衡策略；服务器重启或者宕机不会造成Session丢失；扩展能力强；适合大集群数量使用；

b) 缺点：对应用有入侵，引入了和Redis的交互代码；



c) 使用：

i) pom.xml：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.session</groupId>
```

```
        <artifactId>spring-session-data-redis</artifactId>
    </dependency>
```

ii) 配置Redis：

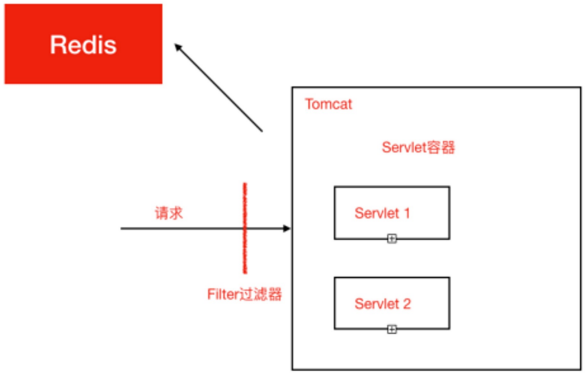
```
spring.redis.database=0
spring.redis.host=127.0.0.1
spring.redis.port=6379
```

iii) 添加注解：

```
@SpringBootApplication
@EnableCaching
@EnableRedisHttpSession
public class LoginApplication  extends SpringBootServletInitializer {
```

d) 原理：

Spring Session原理示意



说明：
请求通过tomcat到达servlet容器的时候，通过过滤器对请求做了一次封装，如果没有过滤器，serlvet就会从tomcat中获取Session
有了过滤器之后，取出来的Session就是Redis中的Session，有的话就从Redis中获取，没有的话就创建并提交到Redis中去